How microGPT Works

Number flow, layers, attention,
and training motivation

J Zhao

1. What the toy GPT is learning
2. Token IDs, embeddings, and positions
3. Attention: Q, K, V and weighted retrieval
4. Transformer layer interpretation
5. Prediction, loss, and Adam update
6. Motivation: why these strange formulas exist

The code trains a character-level autoregressive model.

For a name like `emma`, the sequence is

$$[\mathrm{BOS}, e, m, m, a, \mathrm{BOS}].$$

The model learns conditional probabilities with a finite context window $B = \texttt{block\_size}$:

$$P_\theta\Big(x_{t+1} \mid x_{\max(0,t-B+1)}, \ldots, x_t\Big).$$

Here T is the number of prediction steps for one example. For a name with n characters, $T = n + 1$ because the final BOS/end token is also predicted.
A full name probability is factored as

$$P_\theta(x_1, \ldots, x_T) = \prod_{t=0}^{T-1} P_\theta\Big(x_{t+1} \mid x_{\max(0,t-B+1)}, \ldots, x_t\Big).$$

$$h_t^{(\ell)} = \text{hidden vector at position } t \text{ after layer } \ell.$$

$$h_t^{(0)} = E_{\text{tok}}[x_t] + E_{\text{pos}}[t].$$

discrete text $\longrightarrow$ fixed IDs $\longrightarrow$ learned vectors $\longrightarrow$ probabilities.

The tokenizer gives fixed IDs:

$$a \mapsto 0, \quad b \mapsto 1, \quad \ldots, \quad z \mapsto 25, \quad \mathrm{BOS} \mapsto 26.$$

The token embedding matrix is learned:

$$E_{\mathrm{tok}} \in \mathbb{R}^{27 \times 16}.$$

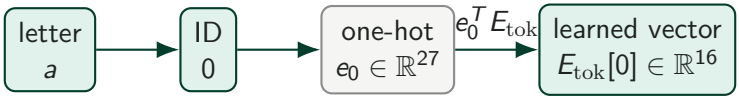
If `m` has ID 12, then

$$E_{\mathrm{tok}}[m] = E_{\mathrm{tok}}[12, :] \in \mathbb{R}^{16}.$$

Initialization:

$$E_{\mathrm{tok}}[i, j] \sim \mathcal{N}(0, 0.08^2).$$

```
matrix = lambda nout, nin, std=0.08: [
    [Value(random.gauss(0, std)) for _ in range(nin)]
    for _ in range(nout)
]
state_dict['wte'] = matrix(vocab_size, n_embd)
```



one-hot vector = a standard basis vector e_i

Every one-hot vector is a unit vector, but not every unit vector is one-hot. The code directly performs the row lookup.

Define

$$\boxed{d := \text{embedding dimension} = \texttt{n\_embd} = 16.}$$

The position embedding matrix is also learned:

$$E_{\text{pos}} \in \mathbb{R}^{B \times d}.$$

In this toy code,

$$B = 16, \qquad d = 16,$$

so

$$E_{\text{pos}} \in \mathbb{R}^{16 \times 16}.$$

This square shape is accidental: context length and embedding width both happen to be 16.

For emma, the input positions are:

position t	input token	target token
0	BOS	e
1	e	m
2	m	m
3	m	a
4	a	BOS

The first m has representation

$$h_2^{(0)} = E_{\text{tok}}[m] + E_{\text{pos}}[2].$$

The second m uses the same token embedding but a different position vector:

$$h_3^{(0)} = E_{\text{tok}}[m] + E_{\text{pos}}[3].$$

The model needs one d-dimensional vector containing both identity and position:

$$h_t^{(0)} = E_{\text{tok}}[x_t] + E_{\text{pos}}[t].$$

Addition is not sacred. It is cheap and keeps dimension fixed.

A more literal alternative would be concatenation:

$$\begin{bmatrix} E_{\text{tok}}[x_t] \\ E_{\text{pos}}[t] \end{bmatrix} \in \mathbb{R}^{2d}.$$

But that widens the model or requires another projection.

At each position, the vector is first normalized:

$$\tilde{h}_t = \mathrm{RMSNorm}(h_t).$$

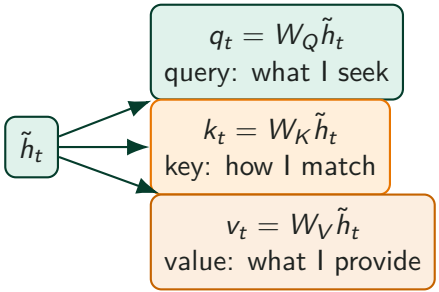
Then three learned linear maps are applied:

$$q_t = W_Q \tilde{h}_t, \qquad k_t = W_K \tilde{h}_t, \qquad v_t = W_V \tilde{h}_t.$$

Here

$$W_Q, W_K, W_V \in \mathbb{R}^{16\times 16}.$$

They are structurally similar, but their roles differ in the attention formula.



Q, K decide routing. V supplies the routed content.

For head dimension

$$d_h = \frac{d}{H} = \frac{16}{4} = 4,$$

the score from current position t to source position s is

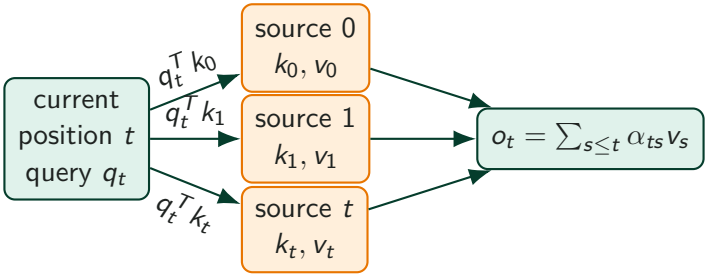
$$s_{ts}^{(h)} = \frac{(q_t^{(h)})^T k_s^{(h)}}{\sqrt{d_h}}.$$

Softmax gives weights:

$$\alpha_{ts}^{(h)} = \frac{e^{s_{ts}^{(h)}}}{\sum_{r \le t} e^{s_{tr}^{(h)}}}.$$

The head output is

$$o_t^{(h)} = \sum_{s \le t} \alpha_{ts}^{(h)} v_s^{(h)}.$$


current
position t
query q_t
$q_t^T k_0$
$q_t^T k_1$
$q_t^T k_t$
source 0
k_0, v_0
source 1
k_1, v_1
source t
k_t, v_t
$o_t = \sum_{s \le t} \alpha_{ts} v_s$

Ignoring transpose conventions, attention is usually written as

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

The attention operation is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_h}}\right) V.$$

In the code, vectors are handled one position at a time with a key-value cache. The mathematics is the same causal version:

$$s \leq t.$$

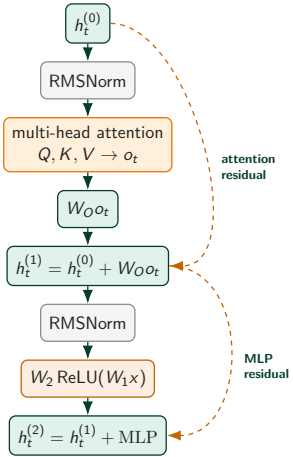
This toy GPT has

$$\texttt{n\_layer} = 1.$$

That one Transformer layer contains:

1. RMSNorm before attention
2. multi-head causal attention
3. output projection W_O
4. residual addition
5. RMSNorm before MLP
6. two-linear-layer MLP: $W_2\,\mathrm{ReLU}(W_1 x)$
7. residual addition

For the MLP, define $d_{\mathrm{ff}} = rd$, where r is the expansion-ratio hyperparameter. Here $r = 4$, independent of the number of attention heads H.


$h_t^{(0)}$
RMSNorm
multi-head attention
$Q, K, V \to o_t$
$W_O o_t$
$h_t^{(1)} = h_t^{(0)} + W_O o_t$
RMSNorm
$W_2\,\mathrm{ReLU}(W_1 x)$
$h_t^{(2)} = h_t^{(1)} + \mathrm{MLP}$
attention residual
MLP residual

Each head returns a vector of dimension

$$d_h = \frac{d}{H}.$$

In this microGPT,

$$d = 16, \qquad H = 4, \qquad d_h = 4.$$

So the four head outputs are

$$o_t^{(1)}, o_t^{(2)}, o_t^{(3)}, o_t^{(4)} \in \mathbb{R}^4.$$

They are joined by concatenation:

$$o_t = \text{Concat}\Big(o_t^{(1)}, o_t^{(2)}, o_t^{(3)}, o_t^{(4)}\Big) \in \mathbb{R}^{4\cdot 4} = \mathbb{R}^{16}.$$

Then the output projection mixes the four coordinate blocks:

$$a_t = W_O o_t.$$

After the heads are concatenated,

$$o_t \in \mathbb{R}^{16}.$$

The output projection mixes heads:

$$a_t = W_O o_t, \qquad W_O \in \mathbb{R}^{16\times 16}.$$

The MLP is

$$\mathrm{MLP}(x) = W_2\,\mathrm{ReLU}(W_1 x).$$

Let $d_{\mathrm{ff}} = rd$ with expansion ratio $r = 4$:

$$W_1 \in \mathbb{R}^{rd\times d} = \mathbb{R}^{64\times 16}, \qquad W_2 \in \mathbb{R}^{d\times rd} = \mathbb{R}^{16\times 64}.$$

$r = 4$ is independent of the number of attention heads H.

The final hidden vector is

$$h_t^{(2)} \in \mathbb{R}^{16}.$$

The output matrix is learned:

$$W_{\text{out}} \in \mathbb{R}^{27\times 16}.$$

It produces logits:

$$\ell_t = W_{\text{out}} h_t^{(2)} \in \mathbb{R}^{27}.$$

logits are raw, unnormalized scores before softmax.

Each coordinate $\ell_{t,i}$ is the raw score for token i.
Softmax produces probabilities:

$$p_{t,j} = \frac{e^{\ell_{t,j}}}{\sum_{k=0}^{26} e^{\ell_{t,k}}}.$$

W_{out} is a parameter. p_t is the prediction.

The target is a discrete token ID:

$$\boxed{y_t = x_{t+1} \in \{0, \ldots, V-1\}.}$$

It is not a vector; its embedding would be $E_{\text{tok}}[y_t] \in \mathbb{R}^d$. The loss at position t is

$$L_t = -\log p_{t,y_t}.$$

For the full sequence,

$$L(\theta) = -\frac{1}{T} \sum_{t=0}^{T-1} \log P_\theta\Big(x_{t+1} \mid x_{\max(0,t-B+1)}, \ldots, x_t\Big).$$

Here T is the number of prediction steps; for a name with n characters, $T = n + 1$. This is cross-entropy / negative log-likelihood.

It is small when the model assigns high probability to the observed next token.

All learned matrices are part of θ:

$$\theta = \{E_{\text{tok}}, E_{\text{pos}}, W_Q, W_K, W_V, W_O, W_1, W_2, W_{\text{out}}\}.$$

For this toy model,

$$\dim(\Theta) = \underbrace{2Vd}_{E_{\text{tok}}+W_{\text{out}}} + \underbrace{Bd}_{E_{\text{pos}}} + \underbrace{12Ld^2}_{\text{attention and MLP matrices}}.$$

For one Transformer layer,

$$12d^2 = \underbrace{4d^2}_{W_Q,W_K,W_V,W_O} + \underbrace{8d^2}_{W_1,W_2}.$$

The MLP part is $8d^2 = 4d^2 + 4d^2$ because $W_1 \in \mathbb{R}^{4d\times d}$ and $W_2 \in \mathbb{R}^{d\times 4d}$.
With $V = 27, B = 16, d = 16, L = 1$:

$$2(27)(16) + 16(16) + 12(1)(16^2) = 4192, \qquad \Theta = \mathbb{R}^{4192}.$$

For each parameter coordinate, let

$$g_t = \frac{\partial L_t}{\partial \theta}.$$

Adam keeps two running estimates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Update:

$$\theta_t = \theta_{t-1} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

The old bottleneck: compress the whole previous sequence into one state.

$$h_t = f(h_{t-1}, x_t).$$

Attention avoids this by keeping all previous position representations and retrieving selectively:

$$o_t = \sum_{s \le t} \alpha_{ts} v_s.$$

The central idea:

do not memorize everything in one vector; retrieve what is relevant.

The model must answer two questions:

Where should I look?

What information should I take?

So it separates matching from content:

$$q_t = W_Q x_t \quad \text{request},$$

$$k_s = W_K x_s \quad \text{index},$$

$$v_s = W_V x_s \quad \text{content}.$$

Then

$$\alpha_{ts} = \text{softmax}_s\left(\frac{q_t^T k_s}{\sqrt{d_h}}\right), \qquad o_t = \sum_s \alpha_{ts} v_s.$$

The dot product is a cheap compatibility score:

$$q_t^T k_s = \|q_t\| \, \|k_s\| \cos\theta.$$

In matrix form, all pairwise scores are computed by

$$QK^T.$$

Softmax converts arbitrary scores into differentiable selection weights:

$$\alpha_s = \frac{e^{s_s}}{\sum_r e^{s_r}}.$$

It is a smooth version of choosing the most relevant source.

If coordinates of q and k have roughly unit variance, then

$$q^T k = \sum_{j=1}^{d_h} q_j k_j$$

has variance about

$$\mathrm{Var}(q^T k) \approx d_h.$$

So the typical scale grows like $\sqrt{d_h}$.
Dividing by $\sqrt{d_h}$ keeps score magnitudes stable:

$$\frac{q^T k}{\sqrt{d_h}}.$$

This prevents softmax from saturating too early and improves gradients.

Multiple heads allow several retrieval patterns in parallel:

$$o_t = o_t^{(1)} \| o_t^{(2)} \| \cdots \| o_t^{(H)}.$$

Then W_O mixes the head outputs.

Attention communicates across positions:

which previous information matters?

The MLP performs nonlinear computation at each position:

how should the gathered information be transformed?

Embeddings create token-position vectors.

Attention retrieves relevant previous information.

The MLP transforms it nonlinearly.

The output head turns it into next-token probabilities.

$$h_t^{(\ell)} \leftrightarrow \texttt{x}, \qquad t \leftrightarrow \texttt{pos\_id}, \qquad \ell \leftrightarrow \texttt{li}.$$

Mathematics	Code
E_{tok}	`state_dict['wte']`
E_{pos}	`state_dict['wpe']`
W_Q	`state_dict[f'layer{li}.attn_wq']`
W_K	`state_dict[f'layer{li}.attn_wk']`
W_V	`state_dict[f'layer{li}.attn_wv']`
W_O	`state_dict[f'layer{li}.attn_wo']`
W_1	`state_dict[f'layer{li}.mlp_fc1']`
W_2	`state_dict[f'layer{li}.mlp_fc2']`
W_{out}	`state_dict['lm_head']`

The code reuses `x`; the mathematical notation keeps layer and position explicit.

The entries are initialized as

$$E_{\text{tok}}[i, j] \sim \mathcal{N}(0, 0.08^2).$$

The value 0.08 is a heuristic, not a mathematical constant. It breaks symmetry while keeping early activations, attention scores, and gradients moderate.

- Too large: dot products $q^T k$ may be large, making softmax too sharp.
- Too small: signals and gradients may be weak.
- More systematic alternatives include Xavier and He initialization.

```
matrix = lambda nout, nin, std=0.08: [
    [Value(random.gauss(0, std)) for _ in range(nin)]
    for _ in range(nout)
]
```

For $x \in \mathbb{R}^d$,

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \varepsilon}.$$

General RMSNorm is

$$\mathrm{RMSNorm}_g(x) = g \odot \frac{x}{\mathrm{RMS}(x)}.$$

Residual additions can cause scale drift:

$$x \leftarrow x + \mathrm{Attention}(x), \qquad x \leftarrow x + \mathrm{MLP}(x).$$

RMSNorm stabilizes attention-score and MLP-input magnitudes. Unlike LayerNorm, it does not subtract the mean.

RMSNorm stabilizes the scale of hidden vectors before learned transformations.

Standard RMSNorm usually learns

$$g \in \mathbb{R}^d, \qquad g_i = 1 \text{ at initialization.}$$

fixed $g = \mathbf{1}$: normalization only	learned g : normalization plus coordinatewise rescaling

This microGPT fixes $g = \mathbf{1}$ implicitly, so g is not a trainable parameter in the actual code.

```
def rmsnorm(x):                  # actual microGPT version
    ms = sum(xi * xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]

# learned-gain version would use:
# return [gi * xi * scale for xi, gi in zip(x, gain)]
```

Fixing g is valid for a tiny model; learned g usually improves flexibility at negligible cost.

For each attention head,

$$q_t, k_t, v_t \in \mathbb{R}^{d_h}, \qquad d_h = \frac{d}{H}.$$

For microGPT,

$$d = 16, \qquad H = 4, \qquad d_h = 4.$$

Thus each head uses

$$q_t, k_t, v_t \in \mathbb{R}^4.$$

Across all heads,

$$Hd_h = d = 16.$$

d determines the total Q, K, V dimension, while $d_h = d/H$ is the dimension per head.

$$\mathrm{MLP}(x) = W_2\,\mathrm{ReLU}(W_1x),$$

where

$$x \in \mathbb{R}^d, \quad W_1 \in \mathbb{R}^{d_{\mathrm{ff}} \times d}, \quad W_2 \in \mathbb{R}^{d \times d_{\mathrm{ff}}}.$$

For microGPT,

$$d = 16, \qquad d_{\mathrm{ff}} = 64.$$

Let

$$z = W_1x \in \mathbb{R}^{64}, \qquad \mathrm{ReLU}(z)_i = \max(0, z_i),$$

then

$$y = W_2\,\mathrm{ReLU}(z) \in \mathbb{R}^{16}.$$

Coordinate form:

$$y_i = \sum_{j=1}^{64} (W_2)_{ij} \max\left(0, \sum_{k=1}^{16} (W_1)_{jk} x_k\right).$$

W_1 expands $16 \to 64$, ReLU adds nonlinearity, W_2 compresses $64 \to 16$.